

Archer Duels — Kompletní technická dokumentace

Dělostřelecký souboj lukostřelců ve stylu Bowmasters (Python + Pygame)

Vygenerováno pro projekt workshop_game

14. června 2026

Obsah

1	Úvod	4
1.1	Co je to za hru	4
1.2	Použité technologie	4
1.3	Filozofie architektury	4
2	Struktura projektu	5
2.1	Co je „balíček“ a „modul“	6
3	Spuštění a celkový tok aplikace	7
3.1	main.py	7
3.2	Životní cyklus na nejvyšší úrovni	7
4	Konfigurace — core/config.py	8
4.1	Cesty k assetům	8
4.2	Displej	8
4.3	Dlaždice / terén	8
4.4	Fyzika	9
4.5	Hratelnost	9
4.6	AI	9
4.7	Barvy	10
5	Načítání obrázků — core/assets.py	11
6	Aplikace a stavový automat — core/application/	12
6.1	state.py	12
6.2	application.py — singleton Application	12
6.2.1	Co je singleton a jak je zde udělán	12
6.2.2	Proč je __init__ „prázdný“	12
6.2.3	Herní smyčka run()	12
6.2.4	Přechody stavů	13
7	Souboj — core/match/	15
7.1	match.py — třída Match	15
7.1.1	Důležité atributy	15

7.1.2	Vznik postav <code>_spawn_archers</code>	15
7.1.3	Fáze a průběh tahu	15
7.1.4	Vyhodnocení dopadu <code>_resolve_projectile</code>	16
7.1.5	Výbuch <code>_explode</code>	16
7.1.6	Zpracování vstupu <code>handle_event</code>	16
7.1.7	Pohyb <code>_handle_walk</code>	17
7.1.8	Tah AI <code>_ai_turn</code>	17
7.1.9	Řízení animací <code>_update_anim_states</code>	17
7.1.10	Aktualizace a vykreslení	17
7.2	<code>explosion.py</code> — třída <code>Explosion</code>	17
8	Procedurální generace terénu — <code>core/terrain/</code>	19
8.1	Co je dlaždice (tile) a čím se liší od spritu	19
8.2	<code>terrain.py</code> — třída <code>Terrain</code>	19
8.2.1	Datové členy	19
8.2.2	Statické pozadí <code>_make_background</code>	19
8.2.3	Generování <code>generate</code>	19
8.2.4	Výškový profil <code>_generate_heights</code> — jádro procedurální generace	20
8.2.5	Kamenné shluky <code>_add_stone</code>	20
8.2.6	Podloží <code>_add_bedrock</code>	21
8.2.7	Vykreslení dlaždic <code>_blit_region</code>	21
8.2.8	Kolizní dotazy	21
8.3	<code>tileset.py</code> — třída <code>Tileset</code>	21
9	Sprity — <code>core/sprites/</code>	22
9.1	<code>sprite_sheet.py</code> — třída <code>SpriteSheet</code>	22
9.2	<code>animated_sprite.py</code> — třída <code>AnimatedSprite</code>	22
9.3	<code>archer.py</code> — třída <code>Archer</code>	22
9.3.1	Geometrie	23
9.3.2	Pohyb <code>walk</code>	23
9.3.3	Výzbroj	23
9.3.4	Zmenšení snímků na velikost lukostřelce	23
9.3.5	Animace podle stavu (řádky sheetu)	23
9.3.6	Kreslení <code>draw_vector</code> (fallback)	24
9.4	<code>player.py</code> a <code>enemy.py</code>	24
9.5	<code>arrow.py</code> a <code>bomb.py</code> — projektily	24
10	Zbraně a fyzika — <code>core/weapons/</code>	26
10.1	<code>weapon.py</code> — typy zbraní	26
10.2	<code>loadout.py</code> — výzbroj	26
10.3	<code>physics.py</code> — jádro fyziky (sdílený zdroj pravdy)	26
10.3.1	Krok integrace <code>step_physics</code>	26
10.3.2	Počáteční rychlost <code>velocity_from_angle_power</code>	26
10.3.3	Míření tažením <code>aim_from_drag</code> (prak)	27
10.3.4	Simulace dráhy <code>simulate_path</code>	27
10.4	<code>projectile.py</code> — základní třída <code>Projectile</code>	27
10.4.1	Let <code>update(dt, terrain, archers)</code>	27
11	Umělá inteligence — <code>core/ai/enemy_ai.py</code>	28

11.1 Řešič <code>aim_solution</code>	28
11.2 Stavový automat <code>take_turn</code>	28
11.3 Zpětná vazba <code>notify_result</code>	28
12 Uživatelské rozhraní — <code>core/ui/</code>	29
12.1 <code>fonts.py</code>	29
12.2 <code>background.py</code>	29
12.3 <code>menu.py</code>	29
12.4 <code>hud.py</code>	29
12.5 <code>overlays.py</code>	29
13 Souřadnice, kolize a klíčové nuance	30
13.1 Dva souřadnicové systémy	30
13.2 Tři druhy kolizí	30
13.3 Proč substepping	30
13.4 Proč ořez <code>dt</code>	30
13.5 Náhodná zbraň každý tah	30
13.6 „Zamčení“ AI	30
14 Jak přidat grafiku (spritesheety a tileset)	31
14.1 Postavy	31
14.2 Projektily	31
14.3 Dlaždice terénu	31
15 Slovníček pojmů	32
16 Fyzika hry do hloubky	33
16.1 Souřadnicový systém a jednotky	33
16.2 Eulerova metoda krok za krokem	33
16.3 Odpor vzduchu u bomby	34
16.4 Počáteční rychlost z míření	34
16.5 Numerický příklad jednoho výstřelu (šíp)	34
16.6 Doba do vrcholu a dostřel (přibližně, bez terénu)	34
16.7 Ochrana proti tunelování (substepping)	35
16.8 Úplný algoritmus letu (<code>Projectile.update</code>)	35
17 Jak se kreslí náhled (vzorec) dráhy	36
17.1 Krok za krokem	36
17.2 Proč náhled přesně sedí	37
17.3 Tentýž nástroj používá i AI	37
18 Závěr	38

1 Úvod

Tento dokument je **úplná technická dokumentace** hry *Archer Duels*. Je psán pro čtenáře, který projekt **vůbec nezná** — vysvětluje tedy nejen *co* jednotlivé soubory dělají, ale i *proč a jak* to dělají, včetně matematických vzorců, postupu procedurální generace terénu, modelu fyziky, chování umělé inteligence a přechodů mezi herními stavy.

1.1 Co je to za hru

Archer Duels je tahový **dělostřelecký souboj** (artillery game) pro jednoho hráče proti počítači, inspirovaný hrou *Bowmasters* a klasikou *Worms*. Dva lukostřelci stojí na náhodně vygenerované krajině. Hráči se střídají v tazích; v každém tahu může postava buď **vystřelit** (lukem šíp, nebo hodit bombu), nebo se **posunout** po terénu. Šíp i bomba letí balistickou dráhou (parabola) pod vlivem gravitace. Bomba navíc **ničí terén** — vytváří kráter — a odhazuje zasažené postavy. Cílem je snížit protivníkovi životy (HP) na nulu.

Hlavní vlastnosti:

- **Procedurálně generovaný terén** — pokaždé jiná krajina z dlaždic (tiles).
- **Destruktivní prostředí** — bomby vyhloubí do hlíny kráter, kámen a podloží zůstávají.
- **Fyzika projektilů** — jednotná simulace sdílená živým výstřelem, náhledem dráhy i řešičem AI.
- **Umělá inteligence** — protivník si dráhu „dostřeluje“ a po zásahu se „zamkne“ na cíl.
- **Tahový systém** s fázemi (míření, vyhodnocení, konec hry).

1.2 Použité technologie

- **Python 3.13**
- **Pygame 2.6.1** — knihovna pro 2D hry (okno, vykreslování, vstup, sprity).
- **NumPy 2.4.6** — rychlé operace nad mřížkou terénu (pole, masky, konvoluce).

Virtuální prostředí je ve složce `archer/`. Hra se spouští příkazem:

```
./archer/bin/python main.py
```

1.3 Filozofie architektury

Projekt byl záměrně **rozdělen do balíčků** (packages) podle herních vzorů: každá třída leží ve vlastním souboru a veřejné rozhraní balíčku se exportuje v souboru `__init__.py`. Díky tomu se importuje čistě, např.:

```
from core.sprites import Player, Enemy, Arrow, Bomb
from core.weapons import simulate_path, aim_from_drag
```

a ne ošklivě jako `core.sprites.player.Player`. Vstupním bodem celé aplikace je jediný **singleton** `Application`, který vlastní okno, herní smyčku a stav. Soubor `main.py` proto obsahuje jen tři řádky.

2 Struktura projektu

Adresářový strom (bez virtuálního prostředí archer/ a složky assets/):

```
main.py                # vstupní bod: application.run()
assets/
  background.png        # statické pozadí (společné pro všechny obrazovky)
core/
  __init__.py
  config.py             # všechny konstanty (rozlišení, fyzika, barvy, cesty)
  assets.py            # načítání a cache obrázků

  application/
    __init__.py        # export: application, Application, ApplicationState
    application.py     # singleton: okno, herní smyčka, přechody stavů
    state.py           # ApplicationState (Enum): SPLASH / MENU / MATCH

  match/
    __init__.py        # export: Match
    match.py           # průběh souboje: tahy, vstup, vyhodnocení výstřelu
    explosion.py       # efekt výbuchu

  terrain/
    __init__.py        # export: Terrain
    terrain.py         # procedurální terén z dlaždic + kolize + ničení
    tileset.py         # volitelná textura dlaždic (jinak výplň barvou)

  sprites/
    __init__.py        # export: Player, Enemy, Archer, Arrow, Bomb, ...
    sprite_sheet.py    # nakrájení spritesheetu na snímky
    animated_sprite.py # přehrávání snímků + vektorový fallback
    archer.py          # základní třída lukostřelce
    player.py          # Player(Archer) – člověk
    enemy.py           # Enemy(Archer) – AI
    arrow.py           # Arrow(Projectile) – šíp
    bomb.py            # Bomb(Projectile) – bomba

  weapons/
    __init__.py        # export: WEAPONS, fyzika, Projectile, ...
    weapon.py          # typy zbraní a registr
    loadout.py         # výzbroj postavy
    physics.py         # fyzika letu + vzorce (sdílený zdroj pravdy)
    projectile.py      # základní třída projektilu (let + kolize)

  ai/
    __init__.py        # export: EnemyAI
    enemy_ai.py        # řešič dráhy + stavový automat míření

  ui/
```

<code>__init__.py</code>	<code># export: Menu, draw_hud, draw_splash, ...</code>
<code>fonts.py</code>	<code># cache fontů + pomocník na vystředěný text</code>
<code>background.py</code>	<code># vykreslení statického pozadí</code>
<code>menu.py</code>	<code># úvodní obrazovka a hlavní menu</code>
<code>hud.py</code>	<code># ukazatele HP, info o tahu, náhled dráhy</code>
<code>overlays.py</code>	<code># konec hry a dialog „opravdu ukončit?“</code>

2.1 Co je „balíček“ a „modul“

- **Modul** = jeden soubor `.py` (např. `physics.py`).
- **Balíček** (package) = složka s `__init__.py` (např. `core/weapons/`). Soubor `__init__.py` se spustí při importu balíčku a typicky jen „přeexportuje“ veřejné třídy/funkce z jednotlivých modulů.

Tento vzor (jedna třída = jeden soubor, veřejné API v `__init__.py`) drží kód přehledný a usnadňuje orientaci: když hledáte třídu `Bomb`, je v souboru `bomb.py`.

3 Spuštění a celkový tok aplikace

3.1 main.py

```
from core.application import application
```

```
if __name__ == "__main__":  
    application.run()
```

`application` je již hotová instance singletonu (vytvořená v `core/application/application.py` na konci souboru). `main.py` jen zavolá její metodu `run()`, která rozběhne celou hru.

3.2 Životní cyklus na nejvyšší úrovni

1. Spustí se `application.run()`.
2. Inicializuje se Pygame, vytvoří se okno 1280×720, hodiny (clock) a hlavní skupina spritů.
3. Spustí se **herní smyčka** — nekonečný cyklus, který každý snímek (frame):
 - a) zpracuje události (vstup), b) aktualizuje stav, c) vykreslí obraz, d) prohodí buffery (`pygame.display.flip()`).
4. Po ukončení se Pygame korektně vypne.

Aplikace se v každém okamžiku nachází v jednom ze tří **stavů**: úvodní obrazovka (SPLASH), menu (MENU), nebo probíhající souboj (MATCH). Přejechy mezi nimi řídí `Application` (viz kapitola o stavovém automatu).

4 Konfigurace — core/config.py

Tento modul neobsahuje žádnou logiku, jen **konstanty**. Je to jediné místo, kde se „ladí“ chování hry. Rozdělení do sekcí:

4.1 Cesty k assetům

```
ASSETS_DIR      = .../assets           # absolutní cesta ke složce assets/
BACKGROUND_IMAGE = "background.png"
PLAYER_SHEET    = "player_sheet.png"   # spritesheet hráče (lukostřelec)
PLAYER_FRAME    = (40, 62)             # nominální velikost buňky v sheetu
ENEMY_SHEET     = "enemy_sheet.png"
ENEMY_FRAME     = (40, 62)
SPRITE_ANIM_FPS = 10                   # rychlost přehrávání animace (snímků/s)
TILES_IMAGE     = None                  # dlaždice zatím barevnou výplní
ARROW_IMAGE     = "arrow.png"           # obrázek šípu
BOMB_IMAGE      = "bomb.png"            # obrázek bomby
# Velikosti vykreslení obrázků projektilů (jen když je *_IMAGE nastaveno).
# Tělo lukostřelce je 24x40 px; tyto boxy jsou záměrně menší.
BOMB_DRAW_SIZE  = 18                   # bomba se vejde do boxu 18x18 px
ARROW_DRAW_SIZE = 34                   # šíp se vejde do boxu 34x34 px
HUD_ICON_SIZE   = 26                   # ikona zbraně vedle popisku v HUD
```

Postavy i **projektily už mají obrázky** (player_sheet.png, enemy_sheet.png, arrow.png, bomb.png). Vektorové kreslení zůstává jako **fallback** pro případ, že by konstanta byla None. Tileset dlaždic zatím nastaven není, takže terén se kreslí **barevnou výplní**.

Zdrojové obrázky jsou velké, proto se **zmenšují** na herní velikost:

- **projektily** — arrow.png/bomb.png (256×256 px) se zmenší do boxů ARROW_DRAW_SIZE/BOMB_DRAW_SIZE, bomba je záměrně **menší než postava**.
- **postavy** — snímky ze spritesheetu se zmenší na **velikost těla lukostřelce** (24×40 px) při načtení zápasu (viz kapitola o archer.py).

4.2 Displej

Konstanta	Hodnota	Význam
SCREEN_W	1280	šířka okna v pixelech
SCREEN_H	720	výška okna v pixelech
FPS	60	cílový počet snímků za sekundu
TITLE	"Archer Duels"	titulek okna
SPLASH_TIME	2.0	délka úvodní obrazovky v sekundách

4.3 Dlaždice / terén

Konstanta	Hodnota	Význam
TILE	8	velikost jedné dlaždice v pixelech
COLS	160	počet sloupců mřížky (SCREEN_W // TILE)
ROWS	90	počet řádků mřížky (SCREEN_H // TILE)
EMPTY	0	prázdná dlaždice (vzduch)
DIRT	1	hlína (zničitelná)
STONE	2	kámen (nezničitelný)
SURFACE_MIN_ROW	30	nejvyšší řádek, kde smí být povrch (ROWS // 3)
SURFACE_MAX_ROW	86	nejnižší řádek povrchu (ROWS – 4)

4.4 Fyzika

Konstanta	Hodnota	Význam
GRAVITY	900.0	tíhové zrychlení (px/s ²) pro šíp
BOMB_GRAVITY_MULT	1.6	bomba je těžší → strmější oblouk
BOMB_DRAG	0.6	vodorovný odpor vzduchu bomby (za sekundu)
POWER_TO_SPEED	6.0	převod „síly tažení“ na rychlost
MAX_POWER	180.0	maximální délka tažení myši (px)

4.5 Hratelnost

Konstanta	Hodnota	Význam
MAX_HP	100	maximální životy
WALK_SPEED	160.0	rychlost chůze (px/s)
WALK_DISTANCE	160.0	max. vzdálenost pohybu za jeden tah
ARROW_DAMAGE	35	poškození šípem
BOMB_MAX_DAMAGE	50	poškození ve středu výbuchu
BOMB_RADIUS_TILES	9	poloměr kráteru/výbuchu v dlaždicích
BOMB_KNOCKBACK	70.0	odhození ve středu výbuchu (px)
BOMB_FUSE	4.0	doba do samovolného výbuchu bomby (s)
SAFE_GROUND_ROWS	8	trvalá spodní vrstva (podloží)
STONE_START_ROW	70	odkud níže se rodí kamenné shluky
MIN_SPAWN_GAP	400	min. vodorovná vzdálenost obou postav

4.6 AI

Konstanta	Hodnota	Význam
AIM_HIT_CHANCE	0.45	šance AI trefit, dokud se „dostřeluje“
AIM_MISS_SPREAD	14.0	rozptyl úhlu/síly při schválném minutí
AI_MOVE_THRESHOLD	24	o kolik se musí cíl pohnout, aby AI znovu zamířila

4.7 Barvy

Sekce COL_* definuje RGB barvy (obloha, hlína, kámen, hráč, nepřítel, text, HP pruhy, tlačítka, výbuch atd.). Používají se při vektorovém kreslení a v UI.

5 Načítání obrázků — core/assets.py

Pomocný modul pro načítání obrázků a jejich zmenšování, se dvěma cache:

```
_image_cache = {}
_scaled_cache = {}

def asset_path(name):
    return os.path.join(C.ASSETS_DIR, name)

def load_image(name, alpha=True):
    if name not in _image_cache:
        img = pygame.image.load(asset_path(name))
        _image_cache[name] = img.convert_alpha() if alpha else img.convert()
    return _image_cache[name]

def load_scaled(name, box, alpha=True):
    key = (name, box)
    if key not in _scaled_cache:
        img = load_image(name, alpha)
        w, h = img.get_size()
        s = min(box / w, box / h) # zachovej poměr stran
        size = (max(1, round(w*s)), max(1, round(h*s)))
        _scaled_cache[key] = pygame.transform.smoothscale(img, size)
    return _scaled_cache[key]
```

Proč cache: stejný obrázek (např. pozadí) může chtít víc míst. Cache zajistí, že se z disku načte jen jednou.

load_scaled(name, box) vrátí obrázek **zmenšený tak, aby se vešel do čtverce** box×box při zachování poměru stran. Výpočet měřítka:

$$s = \min\left(\frac{\text{box}}{w}, \frac{\text{box}}{h}\right), \quad (w', h') = (\lceil w \cdot s \rceil, \lceil h \cdot s \rceil).$$

Použití smoothscale (vyhlazené zmenšení) dává hladký výsledek. Každá velikost se počítá jen jednou (cache podle dvojice (name, box)). Tuto funkci využívají **bomba**, **šíp** i **ikony zbraní v HUD** — proto jsou velké zdrojové PNG (256×256) vykresleny v rozumné herní velikosti.

Co je convert() / convert_alpha(): Pygame umí kreslit nejrychleji, když má obrázek stejný pixelový formát jako obrazovka. convert() obrázek do tohoto formátu převede; convert_alpha() totéž, ale zachová **průhlednost** (alfa kanál — důležité pro sprity s průhledným pozadím). Pozor: tyto funkce vyžadují, aby už existovalo okno (set_mode), proto se obrázky načítají až za běhu, ne při importu.

6 Aplikace a stavový automat — core/application/

6.1 state.py

```
class ApplicationState(Enum):
    SPLASH = auto()  # úvodní obrazovka s titulkem
    MENU = auto()    # hlavní menu (Play / Quit)
    MATCH = auto()   # probíhá souboj
```

Enum je výčtový typ — místo „magických“ řetězců se používají pojmenované hodnoty, což je bezpečnější (překlep se projeví hned jako chyba).

6.2 application.py — singleton Application

6.2.1 Co je singleton a jak je zde udělán

Singleton je návrhový vzor, kdy může existovat **jen jediná instance** třídy. Zde je zajištěn metodou `__new__`:

```
class Application:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Ať zavoláte `Application()` kdekoli, dostanete pořád stejný objekt. Na konci souboru je navíc vytvořena hotová instance `application`, kterou ostatní moduly importují.

6.2.2 Proč je `__init__` „prázdný“

```
def __init__(self):
    if getattr(self, "_ready", False):
        return
    self.screen = self.clock = self.menu = self.match = None
    self.sprites = None
    self.state = ApplicationState.SPLASH
    self.splash_t = 0.0
    ...
```

`__init__` **záměrně nevolá žádnou funkci Pygame** — jen nastaví atributy na `None`. Důvod: aby šel modul bezpečně naimportovat (a testovat), aniž by se otevíralo okno. Veškerá skutečná inicializace probíhá až v `run()`.

6.2.3 Herní smyčka `run()`

```
def run(self):
    pygame.init()
    self.screen = pygame.display.set_mode((C.SCREEN_W, C.SCREEN_H))
    pygame.display.set_caption(C.TITLE)
    self.clock = pygame.time.Clock()
```

```

self.sprites = pygame.sprite.Group()
self.menu = ui.Menu()
self._running = True
while self._running:
    dt = min(self.clock.tick(C.FPS) / 1000.0, 1.0 / 30.0)
    self._handle_events()
    self._update(dt)
    self._draw()
    pygame.display.flip()
pygame.quit(); sys.exit()

```

Klíčové pojmy:

- **clock.tick(FPS)** uspí program tak, aby smyčka neběžela rychleji než 60× za sekundu, a **vrátí počet milisekund** od minulého snímku.
- **dt** (delta time) = doba posledního snímku v **sekundách**. Násobí se jím veškerý pohyb, takže hra běží stejně rychle bez ohledu na FPS.
- **Ořez min(..., 1/30)**: kdyby snímek z nějakého důvodu trval dlouho (např. 0,5 s), velké dt by způsobilo, že projektil „proskočí“ kus mapy. Proto se dt omezí shora na 1/30 s. Fyzika tak zůstane stabilní.
- **pygame.display.flip()** zobrazí na monitor to, co bylo nakresleno do zadního bufferu (double buffering — zabraňuje blikání).
- **sprites** je „hlavní skupina spritů“ (`pygame.sprite.Group`). Drží živé herní objekty (oba lukostřelce a aktuální projektil). Slouží jako kontejner; vykreslování zůstává explicitní (viz `Match.draw`), aby bylo zaručené pořadí.

6.2.4 Přejchody stavů

Celý stavový automat je rozdělen do tří metod: `_handle_events`, `_update`, `_draw`. Každá se chová podle aktuálního `self.state`.

_handle_events — reakce na vstup:

```

for event in pygame.event.get():
    if event.type == pygame.QUIT:                # křížek okna
        if state == MATCH: self.match.request_exit() # zeptat se na potvrzení
        else: self._running = False
    elif state == MENU:
        action = self.menu.handle_event(event)
        if action == "quit": self._running = False
        elif action == "play": self._start_match()
    elif state == MATCH:
        if self.match.handle_event(event) == "menu":
            self._to_menu()

```

_update — posun času:

```

if state == SPLASH:
    self.splash_t += dt
    if self.splash_t >= C.SPLASH_TIME: # po 2 s
        self.state = MENU

```

```
elif state == MATCH:
    self.match.update(dt)
```

`_draw` — vykreslení podle stavu (`draw_splash`, `menu.draw`, nebo `match.draw`).

Přechody (metody `_start_match` a `_to_menu`):

- *SPLASH* → *MENU*: automaticky po uplynutí `SPLASH_TIME`.
- *MENU* → *MATCH*: když `menu` vrátí akci "play" — vytvoří se nový `Match` a do něj se předá hlavní skupina `spritů`.
- *MATCH* → *MENU*: když `Match.handle_event` vrátí "menu" (konec hry nebo potvrzené ukončení) — `match` se zahodí a skupina `spritů` se vyprázdní.
- *MENU/SPLASH* → *konec*: akce "quit" nebo křížek okna ukončí smyčku.

Důležitý princip: **Match ani Menu nevědí nic o přechodech.** Pouze vracejí „záměry“ jako řetězce ("play", "quit", "menu"). O skutečný přechod se stará výhradně `Application`. Tomu se říká oddělení zodpovědností.

7 Souboj — core/match/

7.1 match.py — třída Match

Match je „mozek“ jednoho souboje. Drží terén, obě postavy, AI, aktuální projektil, výbuchy a stav tahu.

7.1.1 Důležité atributy

- `terrain` — instance `Terrain` (vygenerovaná krajina).
- `p1` (`Player`), `p2` (`Enemy`), `archers` = [`p1`, `p2`].
- `ai` — instance `EnemyAI`.
- `current_idx` — index postavy na tahu (0 nebo 1).
- `projectile` — právě letící střela, nebo `None`.
- `explosions` — seznam aktivních efektů výbuchu.
- `phase` — fáze tahu: "aim" | "resolving" | "gameover".
- `dragging`, `walked` — příznaky vstupu hráče.
- `confirm_exit` — zda se zobrazuje dialog ukončení.
- `shooter`, `ai_target`, `target_hp_before` — pomocné pro vyhodnocení a zpětnou vazbu AI.

7.1.2 Vznik postav `_spawn_archers`

```
while True:
    x1 = random.randint(margin, SCREEN_W//2 - 60)
    x2 = random.randint(SCREEN_W//2 + 60, SCREEN_W - margin)
    if x2 - x1 >= MIN_SPAWN_GAP:    # 400 px
        break
p1 = Player(x1, 0, facing=1)
p2 = Enemy(x2, 0, facing=-1)
p1.ground(terrain); p2.ground(terrain)    # „postavit na zem“
```

Postavy se náhodně rozmístí (každá do své poloviny obrazovky) tak, aby mezi nimi byla mezera aspoň 400 px. `ground()` srovná jejich svislou pozici na povrch terénu v daném sloupci.

7.1.3 Fáze a průběh tahu

Hra je **tahová** a používá tři fáze (phase):

1. **aim** — postava na tahu míří. Pokud je to **hráč**, čeká se na vstup (tažení myši = výstřel, klávesy A/D = pohyb). Pokud je to **AI**, po krátké prodlevě `ai_timer` (0,7 s) AI rozhodne (viz `_ai_turn`).
2. **resolving** — projektil letí; každý snímek se aktualizuje a kontrolují kolize. Jakmile dopadne, `_resolve_projectile` vyhodnotí následky.
3. **gameover** — někdo má 0 HP; čeká se na kliknutí/klávesu pro návrat do menu.

Metody řídící tahu:

- **_begin_turn** — začátek tahu: postava dostane **náhodnou zbraň** pro tento tah (`start_turn` volí náhodný slot), vynulují se příznaky pohybu, nastaví se nápověda do HUD a u AI se nastaví `ai_timer` = 0.7.

- **_end_turn** — kontrola konce hry (má někdo 0 HP?). Pokud ano → gameover a určí se vítěz. Jinak se prohodí `current_idx` a začne tah druhé postavy.
- **_fire(angle, power)** — vytvoří projektil:

```
weapon = self.current.selected.weapon
vel = velocity_from_angle_power(angle, power, weapon.kind)
cls = Arrow if weapon.kind == ARROW else Bomb
self.projectile = cls(self.current.muzzle_pos(), vel, weapon, self.current)
self.phase = "resolving"
```

7.1.4 Vyhodnocení dopadu _resolve_projectile

```
p = self.projectile
if p.outcome == "archer" and p.hit_archer:
    p.hit_archer.take_damage(p.weapon.damage)    # přímý zásah šípem
elif p.outcome == "exploded":
    self._explode(p.impact, p.weapon)            # výbuch bomby
    self._settle_archers()                       # srovnat na nový povrch
# zpětná vazba pro AI (trefila / minula?)
if self.shooter.is_ai:
    self.ai.notify_result(self.ai_target.hp < self.target_hp_before)
self._end_turn()
```

outcome je výsledek letu nastavený projektilem (viz Projectile): "archer" (přímý zásah), "terrain" (zaryl se do země), "exploded" (bomba bouchla), "out" (uletěl mimo).

7.1.5 Výbuch _explode

```
radius_px = BOMB_RADIUS_TILES * TILE           # 9 * 8 = 72 px
terrain.destroy_circle(cx, cy, BOMB_RADIUS_TILES) # vyhloubí kráter
explosions.append(Explosion(cx, cy, radius_px))
for a in archers:
    dist = hypot(ax - cx, ay - cy)
    if dist <= radius_px:
        dmg = int(weapon.damage * (1 - dist/radius_px)) # lineární pokles
        a.take_damage(max(1, dmg))
        push = BOMB_KNOCKBACK * (1 - dist/radius_px)    # odhození
        a.x = clamp(a.x + smer*push)
```

Poškození i odhození **lineárně klesají** se vzdáleností od středu výbuchu: ve středu plné, na okraji poloměru nulové. Odhození navíc změní polohu cíle, což později **donutí AI znovu zamířit** (viz AI).

7.1.6 Zpracování vstupu handle_event

Pořadí kontrol je důležité:

1. Pokud běží dialog `confirm_exit`: Enter = zpět do menu, Esc = zrušit, klik na tlačítka *Quit/Stay*.
2. Pokud `phase == gameover`: jakákoli klávesa/klik → "menu".
3. Esc kdykoli jindy → otevře `confirm_exit`.
4. Jinak jen ve fázi aim a jen pro **lidského** hráče:

- `MOUSEBUTTONDOWN` (levé tlačítko) a ještě se nechodilo → `dragging = True`.
- `MOUSEBUTTONUP` → spočítá `aim_from_drag` a pokud `power >= 10`, vystřelí.
- Enter po pohybu → ukončí tah.

7.1.7 Pohyb `_handle_walk`

Drží-li hráč A/D (nebo šipky), postava se pohne o `WALK_SPEED * dt`. Pohyb je **omezen rozpočtem** `WALK_DISTANCE` (160 px) na tah. Jakmile se rozpočet vyčerpá, tah automaticky končí. Uvolnění kláves tah neukončí — lze pokračovat, dokud rozpočet vydrží, nebo tah ukončit Enterem.

7.1.8 Tah `AI_ai_turn`

```
self.ai_timer -= dt
if self.ai_timer > 0: return # krátká „dramatická“ pauza
action = self.ai.take_turn(current, opponent, terrain)
if action["type"] == "shoot":
    self._fire(action["angle"], action["power"])
else:
    self.current.walk(action["dx"], terrain); self._end_turn()
```

7.1.9 Řízení animací `_update_anim_states`

Match každý snímek (na začátku `update`) nastaví oběma postavám příznaky `aiming` a `moving`, podle nichž si pak postava vybere řádek `spritesheetu`:

- výchozí stav je u obou `False` (klid → řádek dle zbraně);
- jen ve fázi `aim` a jen u **postavy na tahu**:
 - je-li to **AI** → `aiming = True` (míří během svého tahu),
 - je-li to **hráč a táhne myší** → `aiming = True`,
 - jinak, drží-li hráč A/D nebo šipky → `moving = True`.

Tím je rozhodování o animaci na jednom místě a samotné sprity zůstávají „hloupé“.

7.1.10 Aktualizace a vykreslení

`update(dt)` posune efekty výbuchu, nastaví stavy animací, aktualizuje animace postav, a podle fáze buď řeší tah (`aim`) nebo posouvá projektil (`resolving`). `draw(screen)` kreslí v pevném pořadí:

1. terén,
2. obě postavy (aktivní má bílý rámeček),
3. projektil,
4. výbuchy,
5. náhled dráhy (jen při tažení),
6. HUD,
7. případně obrazovka konce hry / dialog ukončení.

7.2 `explosion.py` — třída `Explosion`

Krátký vizuální efekt. Drží střed, poloměr a čas `t` (trvání `dur = 0.4 s`). `update(dt)` přičte čas a vrátí `True`, dokud efekt žije. `draw` kreslí rozpínající se kruh, jehož barva slábne:

```
frac = t / dur
r = int(radius * (0.4 + 0.6*frac)) # roste z 40 % na 100 %
col = (255, int(200 - 120*frac), int(60*(1-frac)))
```

8 Procedurální generace terénu — core/terrain/

Toto je jedna z nejzajímavějších částí hry. Terén **není** sada spritů, ale **dvourozměrné pole čísel** (NumPy mřížka), kde každé číslo je materiál dlaždice.

8.1 Co je dlaždice (tile) a čím se liší od spritu

- **Dlaždice** je buňka mřížky `grid[řádek][sloupec]` o velikosti `TILE = 8 px`. Hodnota je materiál: `EMPTY=0` (vzduch), `DIRT=1` (hlína), `STONE=2` (kámen).
- **Sprite** (`pygame.sprite.Sprite`) je samostatný objekt s vlastním obrázkem (`image`) a obdélníkem (`rect`), který se pohybuje a každý snímek se testuje na kolize.

Rozdíl: dlaždice jsou jen **data**. Neexistují jako objekty a neaktualizují se po jedné. Celá viditelná země se **jednou „zapeče“** do jediné plochy (`Surface`) metodou `_blit_region`. Vykreslení je pak jediný `blit` celé plochy — to je extrémně rychlé oproti vykreslování tisíců spritů.

Převod mezi pixely a mřížkou:

$$\text{sloupec} = \left\lfloor \frac{x}{\text{TILE}} \right\rfloor, \quad \text{řádek} = \left\lfloor \frac{y}{\text{TILE}} \right\rfloor.$$

8.2 terrain.py — třída Terrain

8.2.1 Datové členy

- `grid` — pole tvaru `(ROWS, COLS) = (90, 160)`, typ `uint8`.
- `surface` — „zapečený“ obraz scény (pozadí + dlaždice).
- `background` — statický obrázek pozadí, zvětšený na velikost okna.
- `tileset` — volitelná textura dlaždic (jinak `None` = barevná výplň).

8.2.2 Statické pozadí `_make_background`

```
img = load_image(BACKGROUND_IMAGE, alpha=False)
return pygame.transform.scale(img, (SCREEN_W, SCREEN_H))
```

Načte `assets/background.png` a roztáhne ho na celé okno. (Původní verze hry zde procedurálně kreslila oblohu, slunce, hory, mraky a ptáky — to bylo na přání odstraněno ve prospěch jedné statické grafiky.)

8.2.3 Generování `generate`

Postup:

1. Vynuluj mřížku (vše `EMPTY`).
2. Spočítej **výškový profil** povrchu (`_generate_heights`) — pro každý sloupec řádek, kde začíná povrch.
3. Vyplň hlínou vše pod povrchem (maskou).
4. Přidej kamenné shluky (`_add_stone`).
5. Přidej podloží (`_add_bedrock`).
6. „Zapeč“ obraz (`redraw_all`).

Vyplnění hlínou je elegantní jednořádková maska:

```
row_idx = np.arange(ROWS)[: , None]      # sloupcový vektor řádků
mask = row_idx >= heights[None, :]      # True tam, kde je řádek pod povrchem
grid[mask] = DIRT
```

8.2.4 Výškový profil `_generate_heights` — jádro procedurální generace

Toto je nejdůležitější vzorec celé generace. Cílem je získat **přírozeně zvlněnou krajinu** — ne úplně náhodnou (to by byl „šum“), ale ani moc hladkou. Postup kombinuje tři techniky:

1) Náhodná procházka (random walk). Pro každý sloupec vygenerujeme náhodný krok z normálního rozdělení $\mathcal{N}(0, 1)$ a uděláme **kumulativní součet**:

$$\text{steps}_i \sim \mathcal{N}(0, 1), \quad \text{walk}_i = \sum_{j=0}^i \text{steps}_j.$$

Náhodná procházka dává hrubý, „kopcovitý“ tvar, ale je trhaná.

2) Vyhlazení klouzavým průměrem (konvoluce). Trhanost odstraníme průměrováním přes okno šířky $k = 21$ sloupců. To je **konvoluce** s jádrem samých jedniček poděleným k :

$$\text{kernel} = \frac{1}{k} [1, 1, \dots, 1], \quad \text{walk} = \text{walk} * \text{kernel}.$$

V kódu `np.convolve(walk, kernel, mode="same")`. Výsledkem jsou plynulé kopce.

3) Přidání sinusoid. Pro pravidelnější zvlnění se přičtou dvě sinusovky s náhodnou frekvencí a fází. Nechť x je rovnoměrné dělení intervalu $[0, 4\pi]$ na COLS bodů:

$$\text{waves} = 6 \sin(x \cdot U_1 + \varphi_1) + 3 \sin(x \cdot U_2 + \varphi_2),$$

kde $U_1 \in [0,5, 1,5]$, $U_2 \in [1,5, 3,0]$ a fáze $\varphi \in [0, 6]$ jsou náhodné. Druhá vlna má menší amplitudu (3 vs. 6) a vyšší frekvenci — přidá jemnější detail na velké kopce.

4) Normalizace do povoleného pásma. Sečteme profil a převedeme ho do rozsahu $[0, 1]$, pak namapujeme na povolené řádky povrchu:

$$\begin{aligned} \text{profile} &= \text{walk} + \text{waves}, \\ \text{profile} &\leftarrow \frac{\text{profile} - \min(\text{profile})}{\max(\text{profile})}, \end{aligned}$$

`heights = SURFACE_MIN_ROW + profile * (SURFACE_MAX_ROW - SURFACE_MIN_ROW)`.

Nakonec se výsledek ořízne (`np.clip`) do $[30, 86]$ a převede na celá čísla. Povrch tedy nikdy nesáhá výš než třetina obrazovky a nikdy níž než 4 řádky od dna.

8.2.5 Kamenné shluky `_add_stone`

Vygeneruje se 5 až 8 shluků. Každý je **elipsa** se středem (c_x, c_y) a poloosami $r_x \in [8, 22]$, $r_y \in [5, 12]$. Buňka patří do elipsy, pokud:

$$\left(\frac{c - c_x}{r_x} \right)^2 + \left(\frac{r - c_y}{r_y} \right)^2 \leq 1.$$

Na kámen se ale promění **jen už existující hlína** (& `(grid == DIRT)`), takže kámen vzniká pod zemí, ne ve vzduchu. Střed se rodí od řádku `STONE_START_ROW = 70` níže.

8.2.6 Podloží `_add_bedrock`

Spodních `SAFE_GROUND_ROWS` = 8 řádků je natvrdo `STONE`. Tato vrstva se **nikdy nezničí** — zajišťuje, že terén nelze prostřelit skrz naskrz.

8.2.7 Vykreslení dlaždic `_blit_region`

Projde obdélníkovou oblast mřížky a pro každou buňku nakreslí dlaždici do `self.surface`:

- `EMPTY` → vykreslí výřez **pozadí** (aby kráter „prosvítal“ obloha).
- má-li `tileset` → `blit` příslušné textury dlaždice.
- `STONE` → šedá výplň s drobnou texturou.
- `DIRT` → hnědá; pokud je nahoře (nad ní vzduch), dostane **travnatý okraj**.

Po výbuchu se nepřekresluje celá obrazovka, jen **postižený obdélník** (s malým přesahem, aby se opravily travnaté okraje nově odkrytých buněk). To je důležité pro výkon.

8.2.8 Kolizní dotazy

Veškeré kolize terénu jsou **dotazy nad mřížkou**, ne kolize spritů:

- `solid_at(px, py)` → je dlaždice na daném pixelu pevná?

$$\text{grid}[\lfloor y/\text{TILE} \rfloor, \lfloor x/\text{TILE} \rfloor] \neq \text{EMPTY}.$$

Tím projektil každý podkrok testuje, zda narazil do země.

- `surface_y(px)` → pixelová výška nejvyšší pevné dlaždice ve sloupci. Postava „stojí na zemi“ tak, že její `y` se nastaví na `surface_y(x)`. Proto kopíruje terén a nepropadá se.
- `destroy_circle(cx, cy, r)` — výbuch. Vytvoří kruhovou masku $(c - c_x)^2 + (r - c_y)^2 \leq r^2$, ale smaže **jen hlínu nad podložím** (kámen i bedrock zůstávají), pak překreslí postižený obdélník.

Shrnutí: postavy „chodí po terénu“ přes `surface_y`, bomby „ničí“ přes `destroy_circle`, šípy „zasahují“ zem přes `solid_at`.

8.3 `tileset.py` — třída `Tileset`

Volitelná textura dlaždic. Načte obrázek (jednu řadu buněk velikosti `TILE` v pořadí `EMPTY`, `DIRT`, `STONE`) a metodou `subsurface` z něj vyřízne jednotlivé dlaždice. Není-li `TILES_IMAGE` nastaven, `Terrain` použije barevnou výplň.

Co je `subsurface`: vrací „pohled“ (`view`) — malou plochu, která **sdílí pixely** s rodičovskou plochou (bez kopírování paměti). Proto se každý výřez `.copy()`, aby vznikl nezávislý obrázek.

9 Sprity — core/sprites/

9.1 sprite_sheet.py — třída SpriteSheet

Spritesheet je jeden obrázek obsahující **mřížku** snímků: každý **řádek** je jedna animace (sled snímků), sloupce jsou její jednotlivé snímky. Třída ho načte a rozkrájí na seznam řádků (`self.rows`), kde každý řádek je seznam snímků.

Odvození mřížky zaokrouhlením. Reálné sheety nemusí mít přesné rozměry. Proto se počet sloupců a řádků **odvodí zaokrouhlením** z nominální velikosti buňky:

```
self.cols = max(1, round(w / frame_w))    # např. round(484/40) = 12
rows      = max(1, round(h / frame_h))    # např. round(307/62) = 5
```

a skutečné řezy jsou **rovnoměrně rozložené** podíly šířky/výšky, takže zaokrouhlování nikdy „neujede“ napříč listem:

```
for r in range(rows):
    y0, y1 = round(r*h/rows), round((r+1)*h/rows)
    for c in range(cols):
        x0, x1 = round(c*w/cols), round((c+1)*w/cols)
        frame = sheet.subsurface((x0, y0, x1-x0, y1-y0)).copy()
```

Nuance konkrétních sheetů: `player_sheet.png` a `enemy_sheet.png` jsou 484×307 px. Při nominální buňce 40×62 vyjde mřížka **5 řádků × 12 sloupců** (skutečná buňka ≈ 40×61 px). Animace tedy mají po 12 snímcích a přehrávají se rychlostí 10 snímků/s.

9.2 animated_sprite.py — třída AnimatedSprite

Základ všech vizuálních spritů. Drží **řádky animací** (`rows`), index aktivního řádku (`row_index`), rychlost fps a čas `t`.

```
def animate(self, dt):
    if not self.rows: return
    row = self.rows[min(self.row_index, len(self.rows) - 1)]
    self.t += dt
    self.image = row[int(self.t * self.fps) % len(row)]

def draw(self, surface):
    if self.image is not None: surface.blit(self.image, self.rect)
    else: self.draw_vector(surface)    # fallback
```

Nastavením `row_index` se přepíná **která animace** se hraje; `animate` pak v rámci toho řádku cyklicky střídá snímky podle času.

Klíčová myšlenka — vektorový fallback: dokud nejsou k dispozici snímky, `self.image` je `None` a kreslí se metodou `draw_vector`, kterou potomek implementuje pomocí `pygame.draw`. Jakmile spritesheet dodáte, stejný kód začne vykreslovat `self.image`. Není třeba měnit nic dalšího.

9.3 archer.py — třída Archer

Společný základ lidského hráče i AI. Dědí z `AnimatedSprite`.

9.3.1 Geometrie

Postava má rozměry `BODY_W = 24`, `BODY_H = 40`. Souřadnice (x, y) označují **střed chodidel** (spodek). Důležité metody:

- `body_rect()` — obdélník těla spočítaný z (x, y) .
- `muzzle_pos()` → $(x, y - \text{BODY_H})$ — bod, odkud vylétají projektily (rameno).
- `hit_test(point)` → leží bod uvnitř těla? (`rect.collidepoint`).
- `center()` → $(x, y - \text{BODY_H}/2)$ — střed těla (pro výpočet zásahu výbuchem).

9.3.2 Pohyb walk

```
remaining = WALK_DISTANCE - moved_distance
dx = clamp na zbývajícím rozpočtu
target_x = clamp(x + dx, do okrajů obrazovky)
y = terrain.surface_y(target_x)      # srovnat na nový povrch
moved_distance += |skutečný posun|
facing = znaménko dx
```

Postava se tedy posouvá vodorovně, ale svisle vždy „dosedne“ na terén. Rozpočet `moved_distance` hlídá, aby za tah neušla víc než `WALK_DISTANCE`.

9.3.3 Výzbroj

Každá postava má `loadout` (seznam slotů, jeden na každý druh zbraně). Na začátku tahu (`start_turn`) se **náhodně vybere** aktivní slot — hráč tedy v daném tahu dostane buď luk, nebo bombu, ne podle volby, ale náhodně.

9.3.4 Zmenšení snímků na velikost lukostřelce

Snímky ze `spritesheetu` (zdrojová buňka $\approx 40 \times 61$ px) se při vzniku postavy **zmenší na velikost těla** (`BODY_W` × `BODY_H` = 24×40 px) pomocí `smoothscale`. Děje se to **jednou, při načtení zápasu** (v konstruktoru), ne každý snímek — je to tedy levné. Zmenšený snímek se kreslí přesně na `body_rect()`, takže obrázek i kolizní obdélník lícují (a chodidla sedí na terénu).

9.3.5 Animace podle stavu (řádky sheetu)

Každý **řádek** `sheetu` je jiná animace. Postava si každý snímek vybere řádek podle toho, jakou má zbraň a co právě dělá (metoda `_current_row`):

Řádek	Stav	Podmínka
0	klid + luk	nehýbe se, nemíří, zbraň = luk
1	klid + bomba	nehýbe se, nemíří, zbraň = bomba
2	míření + luk	aiming a zbraň = luk
3	míření + bomba	aiming a zbraň = bomba
4	pohyb	moving (má přednost)

Příznaky `aiming` a `moving` nastavuje **Match** každý snímek (metoda `_update_anim_states`, viz kapitola o souboji) — `sprite` sám o průběhu tahu nic neví, jen přehrává řádek, který mu `Match` určí. `update` pak udělá:

```
self.row_index = self._current_row()
self.animate(dt)          # posune snímek v daném řádku
```

Při kreslení se obrázek navíc **zrcadlí** podle směru (`facing < 0` → `pygame.transform.flip`), aby se postava dívala na protivníka.

9.3.6 Kreslení `draw_vector` (fallback)

Když není `spritesheet` (`*_SHEET = None`), postava se kreslí z primitiv: nohy (čáry), plášť (polygon v barvě týmu), hlava s kapucí (kruhy) a luk (oblouk) se šňůrou a paží. Drobné „dýchání“ zajišťuje `bob = sin(anim_t * 5) * 1.5`. Aktivní postava (na tahu) dostane bílý zaoblený rámeček (ten se kreslí i přes `spritesheet`).

9.4 `player.py` a `enemy.py`

Tenké potomci `Archer`:

```
class Player(Archer):
    def __init__(self, x, y, facing=1):
        rows = SpriteSheet(C.PLAYER_SHEET, *C.PLAYER_FRAME).rows if C.PLAYER_SHEET else
        super().__init__(x, y, C.COL_PLAYER, is_ai=False, facing=facing, rows=rows)
```

`Enemy` je obdobný, jen `is_ai=True`, červená barva a `ENEMY_SHEET`. Každá třída si načte svůj `spritesheet` a předá `Archer` jeho **řádky** animací; o zmenšení a výběr řádku se postará základní třída.

9.5 `arrow.py` a `bomb.py` — projektily

Oba dědí z `Projectile` (viz `core/weapons/projectile.py`) a liší se chováním:

Arrow (`kind = ARROW`):

- Při zásahu postavy → výsledek `"archer"` (přímé poškození).
- Při nárazu do terénu → `"terrain"` (zaryje se).
- Nemá zápalnou šňůru.
- Vykreslení: je-li nastaven `ARROW_IMAGE`, načte se přes `load_scaled(ARROW_IMAGE, ARROW_DRAW_SIZE)` (zmenšen do boxu 34 px) a každý snímek se **otočí** podle úhlu letu (`pygame.transform.rotate`) a vykreslí vystředěně. Bez obrázku se kreslí vektorově (čára + hrot + opeření).

Bomb (`kind = BOMB`):

- Při **jakémkoli** dotyku (postava i terén) → `"exploded"`.
- Má **zápalnou šňůru** `BOMB_FUSE = 4` s; po jejím vypršení vybuchne i ve vzduchu (`tick_fuse`).
- Vykreslení: je-li nastaven `BOMB_IMAGE`, načte se přes `load_scaled(BOMB_IMAGE, BOMB_DRAW_SIZE)` — **zmenšen do boxu 18 px, tedy menší než postava** (24×40), aby působil jako házený předmět. Bez obrázku se kreslí vektorově (kruh se šňůrou).

Nuance velikosti: zdrojové PNG (`arrow.png`, `bomb.png`) jsou 256×256 px. Bez zmenšení by zabraly půl obrazovky. Konstanty `ARROW_DRAW_SIZE = 34` a `BOMB_DRAW_SIZE = 18` určují cílovou velikost; bomba je menší, aby vizuálně „seděla“ vedle lukostřelce.

10 Zbraně a fyzika — core/weapons/

10.1 weapon.py — typy zbraní

```
class WeaponType: # kind, name, damage, color, size
WEAPONS = {
    ARROW: WeaponType(ARROW, "Arrow", 35, COL_ARROW, 18),
    BOMB: WeaponType(BOMB, "Bomb", 50, COL_BOMB, 7),
}
```

WeaponType je jen popis (datová struktura). WEAPONS je registr podle druhu.

10.2 loadout.py — výzbroj

WeaponSlot obaluje jednu zbraň. random_loadout() vrátí po jednom slotu pro každý druh zbraně. Aktivní slot se volí každý tah náhodně (v Archer.start_turn).

10.3 physics.py — jádro fyziky (sdílený zdroj pravdy)

Tento modul je **jediný zdroj pravdy** o letu. Stejně funkce používá živý projektil, náhled dráhy i řešič AI — proto náhled přesně odpovídá realitě.

10.3.1 Krok integrace step_physics

Pohyb se počítá **Eulerovou metodou** (jednoduchá numerická integrace): v každém kroku o délce Δt se nejdřív aktualizuje rychlost a pak poloha.

Pro **šíp**:

$$v_y \leftarrow v_y + g \Delta t, \quad x \leftarrow x + v_x \Delta t, \quad y \leftarrow y + v_y \Delta t,$$

kde $g = \text{GRAVITY} = 900$. (Vodorovná rychlost v_x se nemění.)

Pro **bombu** je gravitace silnější a navíc působí vodorovný odpor vzduchu:

$$v_y \leftarrow v_y + g \cdot m \Delta t, \quad v_x \leftarrow v_x (1 - d \Delta t),$$

kde $m = \text{BOMB_GRAVITY_MULT} = 1,6$ a $d = \text{BOMB_DRAG} = 0,6$. Bomba tak padá strměji a vodorovně „doráží“ pomaleji — má kratší, prudší oblouk.

Pozn.: v Pygame roste osa y **dolů**. Kladné v_y tedy znamená pohyb dolů a gravitace se přičítá ke kladnému v_y .

10.3.2 Počáteční rychlost velocity_from_angle_power

Síla tažení se převede na rychlost a rozloží podle úhlu:

$$\text{speed} = \text{power} \cdot \text{POWER_TO_SPEED}, \quad v_x = \cos \alpha \cdot \text{speed}, \quad v_y = \sin \alpha \cdot \text{speed},$$

kde $\text{POWER_TO_SPEED} = 6$. Maximální power je 180, tedy maximální rychlost je $180 \cdot 6 = 1080$ px/s.

10.3.3 Míření tažením `aim_from_drag` (prak)

Mechanika je **prak**: táhne se *od* postavy a střela letí *opačným* směrem. Vstup: `start` (ústí, tj. rameno postavy) a `release` (poloha myši při puštění).

$$\Delta x = \text{release}_x - \text{start}_x, \quad \Delta y = \text{release}_y - \text{start}_y,$$
$$\text{dist} = \sqrt{\Delta x^2 + \Delta y^2}, \quad \text{power} = \min(\text{dist}, \text{MAX_POWER}),$$
$$\alpha = \text{atan2}(-\Delta y, -\Delta x).$$

Znaménka mínus zajistí onen „opačný směr“ praku: čím dál táhnete doleva-dolů, tím silněji letí střela doprava-nahoru. `power` je oříznut na 180.

10.3.4 Simulace dráhy `simulate_path`

Funkce „dopředu přehraje“ tutéž fyziku a vrátí seznam bodů dráhy, dokud střela nenarazí do terénu/postavy nebo neopustí pole. Používá se pro **náhled dráhy** a pro **AI**. Klíčový je **substepping** (podkroky) proti „tunelování“:

```
dt = 1/FPS
speed = hypot(vx, vy)
nsub = max(1, int(speed*dt / (TILE*0.5)) + 1) # víc podkroků = vyšší rychlost
sub = dt / nsub
```

Rychlá střela by za jeden velký krok mohla „přeskočit“ tenkou zeď. Proto se krok rozdělí na tolik podkroků, aby se za podkrok neurazilo víc než půl dlaždice. V každém podkroku se zkontroluje: opustil pole? trefil postavu (`hit_test`)? narazil do terénu (`solid_at`)? První splněná podmínka let ukončí.

10.4 `projectile.py` — základní třída `Projectile`

Společná logika letu pro šíp i bombu. Je to `pygame.sprite.Sprite` (kvůli zařazení do hlavní skupiny), ale kreslení je vlastní.

10.4.1 `Let update(dt, terrain, archers)`

Stejně substepování jako v `simulate_path`. V každém podkroku:

1. `step_physics` posune střelu.
2. Kontrola opuštění pole → `outcome = "out"`.
3. Kontrola zásahu postavy (kromě střelce a mrtvých) → `on_archer_hit()`.
4. Kontrola terénu (`solid_at`) → `on_terrain_hit()`.

Po podkrocích se zavolá `tick_fuse(dt)` (u bomby odpočet šňůry). Metody `on_archer_hit`, `on_terrain_hit`, `tick_fuse` jsou **háčky** (hooks), které potomci `Arrow/Bomb` přepisují — tím se liší chování při zachování společné fyziky. Po dopadu `_finish` nastaví `outcome`, `impact` a odstraní `sprite` ze skupiny.

11 Umělá inteligence — core/ai/enemy_ai.py

AI má dvě části: **řešič dráhy** (najde úhel a sílu) a **stavový automat míření** (rozhoduje, jak přesně střílet).

11.1 Řešič aim_solution

Pro obloukové zbraně se používá **hrubá síla** (brute force): vyzkouší se mřížka kandidátů úhlu a síly a každý se prožene **stejnou** funkcí `simulate_path`:

```
for ang_deg in range(-170, -5, 5):      # úhly po 5°
    for power in range(50, 181, 12):     # síla po 12
        pts, end, hit = simulate_path(start, vel, kind, terrain, archers=[target])
        if hit is target:
            return angle, power, True     # přímý zásah – hotovo
        score = vzdálenost(dopad, střed_cíle)
        ...uchovej nejlepší...
```

Pokud žádný kandidát netrefí přímo, vrátí se ten s **nejmenší vzdáleností dopadu** od cíle. Příznak `hits` je `True`, pokud nejlepší dopad spadl blíž než $2,5 \cdot \text{TILE}$ (tj. 20 px) od cíle.

11.2 Stavový automat take_turn

AI nestřílí hned dokonale — nejdřív se „dostřeluje“:

1. **Kontrola pohybu cíle:** je-li AI „zamčená“ (locked) a cíl se od zámku posunul o víc než `AI_MOVE_THRESHOLD = 24 px`, zámek se zruší (musí mířit znovu).
2. **Najdi řešení** (`aim_solution`). Když žádné není, AI se jen **posune** k cíli a předá tah.
3. **Schválné minuty, dokud není zamčeno:** pokud `not locked` a náhodné číslo $\geq \text{AIM_HIT_CHANCE} = 0,45$ (tedy s ~55% pravděpodobností), přidá se k úhlu i síle náhodný rozptyl $\pm \text{AIM_MISS_SPREAD}$:

$$\alpha \leftarrow \alpha + \text{rad}(U(-14, 14)), \quad \text{power} \leftarrow \max(40, \text{power} + U(-14, 14)).$$

4. Vrátí akci `{"type": "shoot", "angle", "power"}`.

11.3 Zpětná vazba notify_result

Po vyhodnocení výstřelu `Match` zavolá `notify_result(hit)`:

- **Trefa** $\rightarrow$ `locked = True` a uloží se poloha cíle. Příští tah AI vystřelí **přesně** (žádný rozptyl), dokud se cíl nepohne.
- **Minutí** $\rightarrow$ `locked = False`, AI se příště zase „dostřeluje“.

Výsledné chování působí lidsky: AI nejdřív loví správnou dráhu, a když ji najde, „zamkne se“ a trestá hráče, dokud neuhne (pohybem nebo odhozením po výbuchu).

12 Uživatelské rozhraní — core/ui/

12.1 fonts.py

`font(size, bold)` vrací font z cache (aby se stejný font nevytvářel opakovaně). `center_text(screen, text, size, y, color, bold)` vykreslí text vodorovně vystředěný na dané `y`.

12.2 background.py

`draw_background(screen)` vykreslí statické pozadí (zvětšené a uložené v cache). Používají ho **všechny** obrazovky — splash, menu i jako podklad scény — takže je vzhled jednotný a bez animovaných mraků.

12.3 menu.py

`draw_splash(screen)` kreslí úvodní obrazovku (pozadí + velký titulek).

Třída `Menu` představuje hlavní menu:

- **Tlačítko** je `pygame.Rect` + popisek + akce, uložené v seznamu `self.buttons`. V konstruktoru se rozmístí na střed.
- **Najetí myši (hover)**: každý snímek se v `draw` vezme poloha myši a pro každé tlačítko se testuje `rect.collidepoint(mouse)`. Když je `True`, tlačítko se vykreslí zvýrazněnou barvou. To je celé kouzlo podsvícení — test bodu v obdélníku.
- **Kliknutí**: v `handle_event` se chytí `MOUSEBUTTONDOWN` levým tlačítkem a zkontroluje `rect.collidepoint(event.pos)`. Při zásahu se vrátí akce ("play" / "quit"), kterou pak zpracuje `Application`.

12.4 hud.py

- `_hp_bar` kreslí pruh HP (zelený nad 30 %, jinak červený) s číselným popisem.
- `draw_hud` vykreslí oba pruhy HP, informaci o tahu (kdo je na tahu a jaká náhodná zbraň mu padla) a nápovědu dole. **Vedle textu se zbraní se vykreslí i malá ikona zbraně** (obrázek šípu nebo bomby). Ikona se získá pomocníkem `_weapon_icon(kind)`, který přes `load_scaled(name, HUD_ICON_SIZE)` (26 px) vrátí zmenšený obrázek podle druhu zbraně; pokud daná zbraň nemá nastavený obrázek (*_IMAGE is None), ikona se vynechá. Umístí se hned za popisek a svisle se zarovná na střed řádku textu.
- `draw_aim_indicator` kreslí **náhled dráhy**: zavolá `aim_from_drag` na aktuální pozici myši, spočítá rychlost a přes `simulate_path` získá body dráhy. Z nich vykreslí tečky podél prvních ~78 % dráhy (zbytek se schová, aby výsledek nebyl úplně zřejmý). Vedle postavy je i ukazatel síly.

12.5 overlays.py

- `draw_game_over` ztmaví obrazovku a vypíše „You Win!/You Lose!“.
- `draw_confirm_exit + confirm_exit_buttons` vykreslí dialog „Quit to menu?“ s tlačítky *Quit/Stay* a obsluhou Enter/Esc.

13 Souřadnice, kolize a klíčové nuance

13.1 Dva souřadnicové systémy

- **Pixelové** souřadnice — spojitě (float), používá je fyzika a kreslení.
- **Mřížkové** souřadnice — celočíselné (řádek, sloupec), používá je terén.

Převod: `sloupec = x // TILE`, `řádek = y // TILE`. Pozor: osa *y* roste **dolů**.

13.2 Tři druhy kolizí

1. **Projektíl × terén** — `terrain.solid_at(x, y)` (dotaz nad mřížkou).
2. **Projektíl × postava** — `archer.hit_test(point)` (bod v obdélníku těla).
3. **Postava × terén (gravitace)** — postava nikdy nepadá; její *y* se nastaví na `surface_y(x)`.

Žádná z nich nepoužívá `pygame.sprite.spritecollide` — kolize jsou explicitní a levné. Skupina spritů slouží jen jako kontejner živých objektů.

13.3 Proč substepping

Při 60 FPS a rychlosti až 1080 px/s by střela za jeden snímek urazila až 18 px = přes 2 dlaždice. Tenkou zeď by „protunelovala“. Rozdělení snímku na podkroky (po max půl dlaždici) tomu zabrání. Stejný princip je v `simulate_path` i v `Projectile.update`.

13.4 Proč ořez dt

Kdyby okno zamrzlo na 1 s, `dt = 1` by způsobilo obří skok všech objektů a spoustu „protunelování“. Ořez `min(dt, 1/30)` zaručí, že fyzika nikdy nedostane nereálně velký krok (raději hra na okamžik „zpomalí“, než aby se rozbila).

13.5 Náhodná zbraň každý tah

Hráč si zbraň nevybírání — `start_turn` ji losuje. To je záměrný herní prvek: nutí přizpůsobit strategii (luk = přímý zásah, bomba = plošné poškození a ničení terénu).

13.6 „Zamčení“ AI

Po zásahu se AI zamkne na polohu cíle a střílí přesně. Jediná obrana hráče je **pohyb** — buď vlastní chůzí, nebo nechtěně odhozením po výbuchu. Posun nad práh 24 px AI odemkne a donutí znovu se dostřelovat.

14 Jak přidat grafiku (spritesheety a tileset)

Projekt je **připraven** na výměnu vektorové grafiky za obrázky, aniž by se měnil kód. Postup:

14.1 Postavy

Postavy už spritesheety **mají** (player_sheet.png, enemy_sheet.png). Sheet je mřížka, kde **každý řádek je jedna animace** a sloupce jsou její snímky. Pořadí řádků musí být:

1. řádek 0 — **klid + luk**
2. řádek 1 — **klid + bomba**
3. řádek 2 — **míření + luk**
4. řádek 3 — **míření + bomba**
5. řádek 4 — **pohyb**

Konfigurace:

```
PLAYER_SHEET = "player_sheet.png"
PLAYER_FRAME = (40, 62)           # nominální velikost buňky; mřížka se dopočítá zaokrouhlením
SPRITE_ANIM_FPS = 10              # rychlost animace
```

Mřížka se odvodí jako $\text{round}(\text{šířka}/40) \times \text{round}(\text{výška}/62)$ a snímky se **zmenší na velikost těla lukostřelce** (24×40) při načtení zápasu. Chcete-li vrátit vektorové kreslení, nastavte *_SHEET = None. Počet snímků na řádek je libovolný — kód si ho zjistí sám (u dodaných sheetů je to 12 snímků na řádek).

14.2 Projektily

Šíp a bomba už obrázky **mají nastavené**:

```
ARROW_IMAGE = "arrow.png"         # otáčí se podle úhlu letu
BOMB_IMAGE   = "bomb.png"
ARROW_DRAW_SIZE = 34              # cílová velikost (px) – obrázek se zmenší
BOMB_DRAW_SIZE  = 18              # bomba menší než postava (24x40)
```

Obrázky se přes load_scaled automaticky zmenší do daného boxu (zachová se poměr stran). Chcete-li bombu/šíp větší nebo menší, **stačí změnit *_DRAW_SIZE** — ne samotný obrázek. Vrátit se k vektorovému kreslení lze nastavením *_IMAGE = None.

14.3 Dlaždice terénu

1. Vytvořte obrázek s jednou řadou tří buněk 8×8 px v pořadí **EMPTY, DIRT, STONE**.
2. Nastavte TILES_IMAGE = "tiles.png".

Terrain pak místo barevné výplně bude blitovat textury dlaždic.

Tip: nejprve doplňte jeden typ (např. dlaždice), ověřte vzhled, pak postupně přidávejte další. Dokud je konstanta None, platí původní vektorový/barevný fallback.

15 Slovníček pojmů

- **Pygame Surface** — paměťová „plocha“ s pixely; obrazovka i každý obrázek jsou Surface.
- **blit** — zkopírování (vykreslení) jedné plochy na druhou.
- **subsurface** — výřez plochy sdílející pixely s rodičem (rychlé, bez kopie).
- **convert / convert_alpha** — převod obrázku do formátu obrazovky (rychlé kreslení); `convert_alpha` zachová průhlednost.
- **Rect** — obdélník (`x`, `y`, šířka, výška) s metodami jako `collidepoint`.
- **Sprite / Group** — herní objekt s `image` a `rect` / kontejner sprítů.
- **dt (delta time)** — délka snímku v sekundách; násobí se jím pohyb.
- **FPS** — snímky za sekundu (cíl 60).
- **Eulerova integrace** — nejjednodušší numerická metoda pohybu: aktualizuj rychlost, pak polohu.
- **substepping** — rozdělení snímku na menší kroky proti „tunelování“.
- **random walk** — náhodná procházka (kumulativní součet náhodných kroků).
- **konvoluce / klouzavý průměr** — vyhlazení signálu průměrováním přes okno.
- **singleton** — návrhový vzor s jedinou instancí třídy.
- **fallback** — záložní chování, když chybí lepší možnost (zde vektorové kreslení místo obrázku).

16 Fyzika hry do hloubky

Tato kapitola rozebírá fyzikální model podrobněji než přehled u modulu `physics.py` — včetně odvození, číselného příkladu a porovnání šípu a bomby. Veškerá fyzika žije v souboru `core/weapons/physics.py` a používá ji **tři** místa najednou: živý projektil (`Projectile.update`), náhled dráhy (`draw_aim_indicator`) a řešič AI (`aim_solution`). Díky tomu je model **konzistentní** — co vidíte v náhledu, to se i stane.

16.1 Souřadnicový systém a jednotky

- Poloha (x , y) je v **pixelech**, spojitá (`float`).
- Rychlost (v_x , v_y) je v **pixelech za sekundu** (`px/s`).
- Zrychlení (gravitace) je v **px/s²**.
- Čas dt je v **sekundách**.

Důležité: osa y míří **dolů** (typické pro 2D grafiku). Proto:

- kladné v_y = pohyb dolů,
- gravitace přičítá ke v_y kladnou hodnotu (táhne dolů),
- „nahoru“ je záporné v_y , a při míření je `angle` typicky záporný úhel (proto AI zkouší úhly od -170° do -5° — tedy „doleva nahoru“ až „doprava nahoru“).

16.2 Eulerova metoda krok za krokem

Spojité pohyby popisují rovnice:

$$\frac{dx}{dt} = v_x, \quad \frac{dy}{dt} = v_y, \quad \frac{dv_y}{dt} = g.$$

Počítač je řeší **diskrétně** po malých krocích Δt . Hra používá **semi-implicitní (symplektickou) Eulerovu metodu** — nejdřív se aktualizuje **rychlost**, teprve pak **poloha** s už novou rychlostí:

$$v_y^{\text{nové}} = v_y + g \Delta t, \quad x^{\text{nové}} = x + v_x \Delta t, \quad y^{\text{nové}} = y + v_y^{\text{nové}} \Delta t.$$

Tato varianta je pro hry stabilnější a lépe zachovává tvar dráhy než „obyčejný“ Euler (kde by se poloha posunula starou rychlostí). Přesně to dělá funkce `step_physics`:

```
def step_physics(pos, vel, kind, dt):
    x, y = pos; vx, vy = vel
    if kind == ARROW:
        vy += GRAVITY * dt                                # nejdřív rychlost
    elif kind == BOMB:
        vy += GRAVITY * BOMB_GRAVITY_MULT * dt
        vx *= (1.0 - BOMB_DRAG * dt)                      # vodorovný odpor
    x += vx * dt                                           # pak poloha
    y += vy * dt
    return (x, y), (vx, vy)
```

16.3 Odpor vzduchu u bomby

Bomba má vodorovný odpor: každý krok se v_x vynásobí faktorem $(1 - d \Delta t)$, kde $d = 0,6$. To je diskrétní obdoba **exponenciálního útlumu**. Po čase t (mnoho malých kroků) platí přibližně:

$$v_x(t) \approx v_x(0) \cdot e^{-dt}.$$

Vodorovná rychlost tedy plynule slábne — bomba „nedoletí“ tak daleko jako šíp se stejným nástřelem a její dráha je kratší a strmější. Svisle navíc padá rychleji (gravitace $\times 1,6$). Výsledkem je výrazně jiný „pocit“ obou zbraní.

16.4 Počáteční rychlost z míření

Z úhlu α a síly power vznikne počáteční rychlost:

$$\text{speed} = \text{power} \cdot 6, \quad v_x = \cos \alpha \cdot \text{speed}, \quad v_y = \sin \alpha \cdot \text{speed}.$$

Protože $\text{power} \in [0, 180]$, je rychlost $\in [0, 1080]$ px/s.

16.5 Numerický příklad jednoho výstřelu (šíp)

Mějme $\text{power} = 100$ a úhel $\alpha = -45^\circ$ (doprava nahoru). Pak:

- $\text{speed} = 100 \cdot 6 = 600$ px/s,
- $v_x = \cos(-45^\circ) \cdot 600 \approx 424$ px/s,
- $v_y = \sin(-45^\circ) \cdot 600 \approx -424$ px/s (nahoru).

Krok $\Delta t = 1/60 \approx 0,0167$ s, $g = 900$. První krok (semi-implicitní Euler):

- $v_y \leftarrow -424 + 900 \cdot 0,0167 \approx -409$ px/s,
- $x \leftarrow x + 424 \cdot 0,0167 \approx x + 7,1$ px,
- $y \leftarrow y + (-409) \cdot 0,0167 \approx y - 6,8$ px (stoupá).

S každým krokem v_y roste o ~ 15 px/s, takže šíp zpomaluje stoupání, dosáhne vrcholu (když $v_y = 0$) a začne padat. Vznikne **parabola**.

16.6 Doba do vrcholu a dostřel (přibližně, bez terénu)

Pro **šíp** (bez odporu) platí klasické vztahy šikmého vrhu. Vrcholu dosáhne, když $v_y = 0$:

$$t_{\text{vrchol}} = \frac{|v_{y0}|}{g}.$$

Pro náš příklad $t_{\text{vrchol}} = 424/900 \approx 0,47$ s. Celková doba letu po stejnou výšku je dvojnásobek a vodorovný dostřel:

$$R = v_x \cdot 2 t_{\text{vrchol}} = \frac{v_x \cdot 2 |v_{y0}|}{g} \approx \frac{424 \cdot 848}{900} \approx 399 \text{ px}.$$

(V reálu let ukončí dřív terén nebo zásah; u bomby navíc odpor dostřel zkracuje.)

16.7 Ochrana proti tunelování (substepping)

Při 60 FPS a rychlosti 1080 px/s urazí střela až 18 px za snímek — to jsou **přes dvě dlaždice** (TILE = 8). Bez opatření by mohla „přeskočit“ tenkou zeď a proletět skrz. Řešení: snímek se rozdělí na podkroky tak, aby se za podkrok neurazilo víc než **půl dlaždice**:

$$n_{\text{sub}} = \max\left(1, \left\lfloor \frac{\|v\| \cdot \Delta t}{\text{TILE} \cdot 0,5} \right\rfloor + 1\right), \quad \text{sub} = \frac{\Delta t}{n_{\text{sub}}},$$

kde $\|v\| = \sqrt{v_x^2 + v_y^2}$. Čím rychlejší střela, tím víc podkroků. V každém podkroku se volá `step_physics` a hned se testují kolize.

16.8 Úplný algoritmus letu (`Projectile.update`)

spočítej `nsub` a `sub = dt / nsub`

opakuj `nsub`-krát:

```
(pos, vel) = step_physics(pos, vel, kind, sub)
angle = atan2(vy, vx) # orientace pro vykreslení
když pos mimo pole:   výsledek "out";         konec
pro každou postavu (kromě střelce a mrtvých):
    když hit_test(pos): výsledek on_archer_hit(); konec
    když terrain.solid_at(pos): výsledek on_terrain_hit(); konec
po podkrocích: tick_fuse(dt) # bomba odečte zápalnou šňůru
```

Háčky `on_archer_hit` / `on_terrain_hit` / `tick_fuse` se liší podle zbraně:

Událost	Šíp (Arrow)	Bomba (Bomb)
zásah postavy	"archer" (přímé poškození)	"exploded"
náraz do terénu	"terrain" (zaryje se)	"exploded"
zápalná šňůra	žádná	po 4 s "exploded" ve vzduchu

17 Jak se kreslí náhled (vzorec) dráhy

Náhled dráhy je „pomůcka pro míření“ — tečkovaná čára, která ukazuje, kam střela poletí. Vykresluje se **jen** během tažení myši u lidského hráče. Veškerá logika je ve funkci `draw_aim_indicator` v souboru `core/ui/hud.py`, která se volá z `Match.draw`:

```
if self.phase == "aim" and not self.current.is_ai and self.dragging:
    ui.draw_aim_indicator(screen, self.current,
                          pygame.mouse.get_pos(), self.terrain, self.archers)
```

17.1 Krok za krokem

```
def draw_aim_indicator(screen, archer, mouse_pos, terrain, archers=None):
    start = archer.muzzle_pos() # 1) odkud
    angle, power = aim_from_drag(start, mouse_pos) # 2) míření z tažení
    if power <= 0: return
    kind = archer.selected.weapon.kind
    vel = velocity_from_angle_power(angle, power, kind) # 3) počáteční rychlost
    pts, _, _ = simulate_path(start, vel, kind, terrain, # 4) celá dráha
                        archers=archers, ignore=archer, max_points=160)
    visible_count = max(5, int(len(pts) * 0.78)) # 5) jen 78 % bodů
    for i, p in enumerate(pts[:visible_count]): # 6) tečkovat
        if i % 2 == 0:
            pygame.draw.circle(screen, COL_AIM, (int(p[0]), int(p[1])), 2)
    frac = power / MAX_POWER # 7) ukazatel síly
    pygame.draw.rect(...) # pruh u postavy
```

Rozbor jednotlivých kroků:

1. **Výchozí bod** `start = muzzle_pos()` = rameno postavy (`x`, `y - BODY_H`). Náhled tedy vychází ze stejného místa jako skutečný výstřel.
2. **Míření z tažení** — `aim_from_drag(start, mouse_pos)` spočítá `angle` a `power` z vektoru mezi ramenem a **aktuální** polohou myši. Protože se volá každý snímek s `pygame.mouse.get_pos()`, náhled se **živě** mění, jak táhnete.
3. **Počáteční rychlost** — `velocity_from_angle_power` převede `angle+power` na (`vx`, `vy`). Důležité: zohledňuje **druh zbraně** (`kind`), takže náhled bomby bude strmější než náhled šípu.
4. **Celá dráha** — `simulate_path` „dopředu přehraje“ **úplně stejnou** fyziku (`step_physics`, stejné substepování) jako skutečný výstřel a vrátí seznam bodů `pts`, dokud střela nenarazí do terénu/postavy nebo neopustí pole. Limit `max_points = 160` ji zastaví u extrémně dlouhých drah. Argument `ignore=archer` zajistí, že náhled neukončí kolize s **vlastní** postavou.
5. **Zkrácení na 78 %** — záměrně se vykreslí jen prvních ~78 % bodů (`visible_count = max(5, int(len(pts) * 0.78))`). Konec dráhy (přesné místo dopadu) se **schová**, aby hra nebyla triviální — hráč vidí směr a oblouk, ale ne úplně přesný cíl. Spodní mez 5 bodů zajistí viditelný náhled i u krátkých tahů.
6. **Tečkování** — vykreslí se malý kruh (poloměr 2 px, barva `COL_AIM` = bílá) jen pro **každý druhý** bod (`if i % 2 == 0`). Tím vznikne **přerušovaná** (tečkovaná) čára místo plné — čitelnější a vzdušnější.
7. **Ukazatel síly** — vedle postavy se nakreslí malý pruh, jehož vyplnění odpovídá `power / MAX_POWER` (0–100 %). Hráč tak vidí, jak silný výstřel chystá, nezávisle na tvaru dráhy.

17.2 Proč náhled přesně sedí

Klíčová vlastnost: náhled a skutečný výstřel sdílejí **tytéž funkce** (`aim_from_drag`, `velocity_from_angle_power`, `step_physics`, `substepping` i kolizní dotazy `solid_at/hit_test`). Není to tedy oddělená aproximace — je to **stejná simulace**, jen spuštěná dopředu a useknutá na 78 %. Proto kam míří tečky, tam střela skutečně poletí (až na schovaný konec).

17.3 Tentýž nástroj používá i AI

Funkci `simulate_path` nevyužívá jen náhled, ale i řešič AI (`aim_solution`), který přes ni zkouší stovky kombinací úhlu a síly a hledá zásah. AI i hráč tak „vidí“ svět stejnou fyzikou — jen hráč vizuálně (tečky), AI numericky (skóre vzdálenosti dopadu).

18 Závěr

Archer Duels je kompaktní, ale poučný projekt, který ukazuje řadu klasických herních technik: herní smyčku s dt, stavový automat, procedurální generaci terénu (random walk + konvoluce + sinusoidy), destruktivní mřížkový terén, balistickou fyziku s ochranou proti tunelování a jednoduchou, ale přesvědčivou AI s „dostřelováním“ a zamykáním na cíl.

Architektura je rozdělena do přehledných balíčků s jednou třídou na soubor a čistým veřejným rozhraním v `__init__.py`. Vstupním bodem je singleton `Application`, takže `main.py` obsahuje jen volání `application.run()`. Grafika je připravena na výměnu za spritesheety a tileset bez zásahu do logiky, díky vektorovému (a barevnému) fallbacku.

Tato dokumentace by měla nově příchozímu stačit k tomu, aby projektu rozuměl, dokázal se v něm orientovat a bezpečně ho rozšiřovat.